

Exploring Quantum Machine Learning for Explainable Malware Detection

Giovanni Ciaramella*, Fabio Martinelli*, Francesco Mercaldo[†]*, Antonella Santone[†]

*Institute for Informatics and Telematics, National Research Council of Italy (CNR), Pisa, Italy

g.ciaramella1@studenti.unimol.it, {fabio.martinelli, francesco.mercaldo}@iit.cnr.it

[†]University of Molise, Campobasso, Italy

{francesco.mercaldo, antonella.santone}@unimol.it

Abstract—In recent times is increasing the number of cyberattacks. Daily, we employ computers and mobile devices to perform different tasks, and malicious users are always ready to perpetrate malicious actions. Usually, they use malicious applications able to gain personal information. To avoid these troubles researchers are always focused on the design of new ways to detect malware, typically exploiting machine learning. The progress and recent increasing adoption of quantum computing allowed the inclusion of quantum algorithms into machine learning methods. In this paper, we present a malware detection method based on a quantum machine learning model. In addition, to understand the explainability we adopted two different Class Activation Mapping algorithms to highlight the areas of the images under study that are responsible for a specific categorization of the malware's families. Moreover, we also compare the best results obtained using the quantum model we propose with one of the first convolutional neural network models, i.e., the Le-Net one, to understand whether quantum models can be promising in the malware detection task.

I. INTRODUCTION

Over the years, the number of threats has increased significantly with the widespread of computers and the Internet. In particular, take hold of the diffusion of malicious software, also known as *malware*. The latter term refers to any intrusive software designed and built by a cybercriminal run on the users' machines without their consent. Nowadays, the word represents a term to embed many malicious payloads such as *viruses*, *Trojan horses*, *ransomware*, *spyware*, *adware*, *rogue software*, *wipers*, *scareware*, and each other that have different scopes once executed [1], [2]. As reported in the study published by *Statista*¹ in 2022, the number of attacks has always been very high, reaching a peak in 2018 with 10.5 billion of attacks; meanwhile, only in the first half of the year of 2022, 2.8 billion attacks have registered. Over the years, the attacks trend has stressed the security system built to protect sensitive information or classified data. Despite massive growth in the mobile sector, personal computers remain the chosen target. In a report published by AV-ATLAS², i.e., the web portal of the AV-TEST Institute, the most affected operating system turns out to be Windows. As a result of the above, security experts started to design and implement numerous countermeasures to protect users and prevent the future spread of attacks. One of

the most used techniques is the signature matching approach. The latter is a commonly utilized technique for identifying potential malware, and it needs the use of a database that records signatures and then compares them to prospective threats [3], [4]. Thanks to the progress, researchers have expanded research domains to discover malware. One of the earliest introduced was static analysis [5], which is the automatic study of source code without running the program [6]. Machine Learning has also recently been introduced into the field of security. Due to the novelty of this method, researchers have several opportunities to build models that can detect malware [7]. In particular, security experts started to base their analysis on feature extraction such as *N-gram*, *Metadata*, *Entropy*, *Image representation*, *String length*, *Opcode*, *File size* and many more [8], [9]. With time the threats to software security have evolved gradually, and the number of data is grown enormously; Due to the latter, the hardware limitations, and the progress of research and technology, it was possible to add quantum theory to the machine learning field [10]. In particular, quantum models have been recently used to identify malware in the Android environment [11]. Starting from these considerations, in this paper we present a malware detector method based on a quantum machine learning model. In particular, we design a quantum model i.e., the Hybrid Quantum Model, aimed to simulate the computation performed by a quantum computer, where a first layer of the classical convolutional network is added. Furthermore, we also propose an explainability study that aims to show the features that most influence the model's decision. In detail, we applied two Class Activation Mapping (CAM) algorithms: GradCam++ [12], and ScoreCam [13].

The rest of the paper proceeds as follows: the next section reports the state-of-the-art general usage of deep learning and quantum computing with particular regard to malware detection; in Section III methodology proposed is shown; the results of the experimental analysis in Section IV are presented and, finally, in the last section conclusions and future research plans are drawn.

II. RELATED WORK

In the last period, Deep Learning has taken hold in several fields, particularly malware detection. In addition, with the progress recently also quantum theory has been added to

¹<https://www.statista.com/statistics/873097/malware-attacks-per-year-worldwide/>

²<https://portal.av-atlas.org/malware>

the machine learning field. The latter in different areas has employed.

In health, quantum techniques are widely adopted to identify possible diseases. In 2021, Amin *et al.* [14] introduced Quantum Machine Learning to recognize infection in its early stages increased. Results obtained by the researchers are satisfactory, and in detail, they highlight that QML performs better than Classical Machine Learning.

Authors in [15] focused on malware detection in the Android environment. In particular, in that research, a comparison method between different CNNs, and hybrid and full quantum models is presented. Specifically, they employed a dataset of Android malware to execute experiments, and once identified the best models applied on them the Gradient-weighted Class Activation Mapping. As a result, they achieved that the Convolutional Neural Networks perform better than the Hybrid QCNN. In addition, researchers also highlighted that due to the huge amount of computational resources required, they are not allowed to train the models with the same image size. Different from them, in our work we proposed a comparison between one of the first CNN and the Hybrid QCNN training the models using a small image size. Furthermore, we also showed the usage of two different Gradient-weighted Class Activation Mapping.

Researchers in [16] presented a malware classification algorithm employing particular static features called MCSC (Malware Classification using SimHash and CNN). In particular, to retrieve a grayscale image SimHash technique is used on the disassembled malware, and to identify the specific family, a Convolutional Neural Network was adopted. The authors achieved high classification accuracy on a malware dataset of 10,805 samples.

In [17], a Deep Learning-based malware extraction approach based on opcodes is provided. In particular, the authors adopted the Gradient Boosting algorithm to classify and also employed k-fold cross-validation to validate the model performances.

Researchers in [18] presented malware detection based on a deep learning network using two-dimensional binary program features. In that, they used a database of 400,000 software binary also describing a non-parametric method for adjusting the classifier's scores.

He *et al.* in [19] proposed an investigation of the effectiveness of Convolutional Neural Networks against redundant API injection. To do this, they created a malware detection system capable of converting malware files into grayscale images and classifying them using CNN built utilizing spatial pyramid pooling layers.

III. THE METHOD

In this Section, we present the method we propose aimed at classifying different families of malware using a Quantum Hybrid network. As shown in Figure 1 the process is composed of three steps:

```

NEW_SIZE = 10
[...]
model = models.Sequential()
model.add(QConv(filter_size=2, depth=8, \
    activation='relu', name='qconv1', \
    input_shape=(NEW_SIZE, NEW_SIZE, self.channels)))
model.add(layers.Conv2D(16, (2, 2), \
    activation='relu'))
model.add(layers.Flatten())
model.add(layers.Dense(32, activation='relu'))
model.add(layers.Dense(self.num_classes, \
    activation='softmax'))

```

Listing 1: Structure of the proposed Hybrid Convolutional Neural Network.

A. Dataset Composition

The first step to building a malware detector based on images is the usage of a dataset composed of malware converted into grayscale images. The selection of a robust dataset is essential for achieving adequate training and, consequently, promising outcomes during the classification phase. In this case, we used a dataset consisting of thousands of samples divided into different families, which allowed us to reach a good variety of data. Employing a large number of samples, well labeled, is possible to avoid the overfitting of the model used.

B. Hybrid Quantum Model

The second phase of our proposed methodology is the submission of the samples to a deep-learning network to obtain a model. In this case, we employed a Hybrid Quantum Convolutional Neural Network model (Hybrid-QCNN). Hybrid-QCNNs are meta-networks composed of quantum and classical neural network-based function blocks connected in a directed graph topology [20]. We consider this a rendition of a hybrid quantum-classical computational graph where the inner workings of various functions are abstracted into boxes. The edges represent the flow of classical information through the meta-network of a quantum and classical function. In Listing 1 is reported as the script employed to perform experiments. As we can see, the first layer added is a convolutional layer needed to simulate the calculation performed by a quantum computer.

C. Experiments

As shown in Figure 1, the last step in our proposed method, we performed several experiments. In each of them, we used different settings, and the best performance was obtained with the settings shown in Table I. In addition, we applied two different CAM algorithms to the best-performing model, shown below. **ScoreCam and GradCam++:** Over the years, to help users understand which sections of an image subjected to a Deep Learning model are most interesting, researchers introduced CAMs, a series of algorithms devoted to allowing inspecting images to classify them and figure out which parts/pixels of that image contributed most to the final output of the model. The idea is to use a heat map generated using the gradients

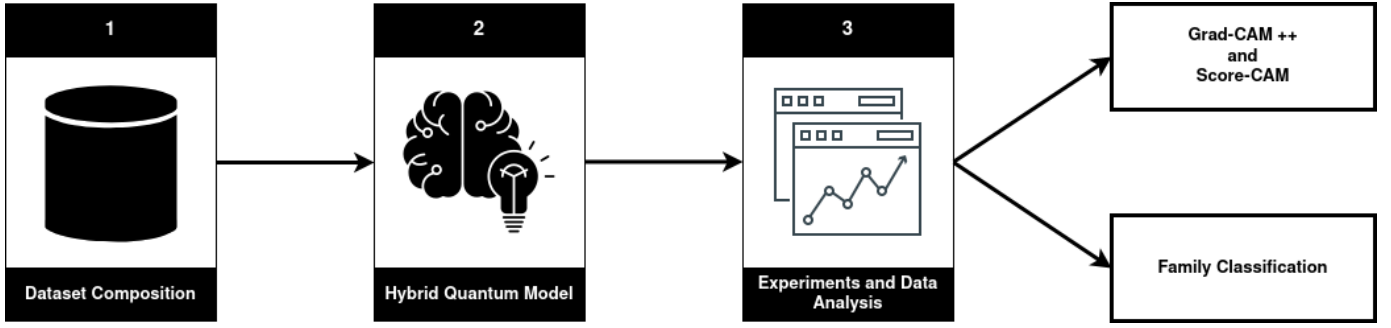


Fig. 1: The proposed approach for explainable quantum malware detection.

of any target concept. Of course, several alternatives have been presented over the years, for instance, the GradCam++ introduced in 2018 by Chattopadhyay *et al.* in [12]. GradCam++ uses the positive partial derivatives of the final convolutional layer feature maps at a given class score as weights to create a visual explanation of the class label under consideration. A different implementation of CAM is the ScoreCam algorithm. The latter has been introduced by Wang *et al.* in [13] and differs from the other techniques because in that gradients are not considered. In particular, according to the study, the propagated gradients are unstable and induce random noise in gradient-based saliency maps. The weights of the score achieved for a specific target class are employed in ScoreCam. As a result, ScoreCam is less dependent on the gradient and may function as a more broad framework because it just requires access to the model's class activation map and output score.

IV. RESULTS

In this section, we describe the results of the experimental analysis we conducted. In particular, in Section IV-A, we describe in detail the dataset and the results obtained. In addition, we also compare the Hybrid QCNN with the Le-Net model. The latter represents one of the first CNN introduced in the world of Deep Learning. This comparison aims to understand the evolution of Deep Learning over the years. On the other hand, Section IV-B, presents the application of two different CAM algorithms to show which areas of an image are most distinctive using the Hybrid Quantum Model. The experiments reported in this paper have been executed using the following hardware and software characteristics: Intel Xeon Gold 6140M CPU at 2.30GHz, 64GB RAM, and Ubuntu 22.04.01 LTS distro as Operating System.

A. Model Evaluation

To perform our experiments we needed to use a dataset. The latter was taken on the Malevis webpage³, and it is composed of 14,226 malware files collected by the Multimedia Information Lab of Hacettepe University Computer Engineering cooperated with Comodo Inc⁴, which have been converted into

grayscale images using a script designed and developed by Sultanik and freely available for research purposes⁵. The data used in this paper are belonging to 25 different families of malware such as *Adposhel* (494), *Agent* (470), *Allaple* (478), *Amonetize* (497), *Androm* (500), *Autorun* (496), *BrowseFox* (493), *Dinwod* (499), *Elex* (500), *Expiro* (501), *Fasong* (500), *HackKMS* (499), *Hlux* (500), *Injector* (495), *InstallCore* (500), *MultiPlug* (499), *Neoreklami* (500), *Neshta* (497), *Regrun* (485), *Salicy* (499), *Snarosite* (500), *Stantinko* (500), *VBA* (500), *VBKrypt* (496), and *Vilse* (496). In addition to the previous, 1832 trusted samples have also been considered. Once we obtained the dataset we divided it into training, validation, and testing, with a percentage of 80-10-10. Specifically, the samples employed are divided as follows: 11374 for the training phase, 1413 for the validation, and 1439 for the test phase.

TABLE I: Hyperparamters adopted to train models

Model	Image Size	Epochs	Batch Size	Training Time HH:MM:SS
Hybrid QCNN	25x1	10	32	11:55:10
		30		1 Day 5:15:40
		10	64	11:54:05
		20		17:44:48
Le-Net	32x1	10	32	0:00:33
		30		0:01:28
		10	64	0:01:06
		20		0:00:46

TABLE II: Results obtained after test set execution

Model	Accuracy	Precision	Recall	F-measure	AUC
Hybrid QCNN	0.850	0.892	0.830	0.860	0.974
	0.819	0.826	0.812	0.819	0.945
	0.771	0.904	0.710	0.795	0.972
	0.824	0.887	0.779	0.829	0.974
Le-Net	0.128	0	0	0	0.558
	0.698	0.904	0.553	0.686	0.970
	0.521	0.834	0.290	0.430	0.933
	0.516	0.830	0.245	0.378	0.927

Table I reports hyperparameters (*i.e.*, *input image size*, *epochs number*, and *batch size*) and the time to train each of the two models performed. Specifically, for each experiment executed, we employed the learning-rate set at 0.001. Due to

³<https://web.cs.hacettepe.edu.tr/~selman/malevis/index.html>

⁴<https://www.comodo.com/>

⁵<https://github.com/ESultanik/bin2png>

computational resources, the Hybrid QCNN model was trained using an image size of $25 \times 25 \times 1$. To compare this model with one of the first CNN models, *i.e.*, *Le-Net*, we also reduced the image size ($32 \times 32 \times 1$) to train both models over the same dataset and with similar hyper-parameters. Table II shows the results for each experiment. In particular, we obtained the best results using the hyperparameters reported in Table II using the first experiment performed with the hybrid model, retrieving an accuracy value of 0.850, a precision of 0.892, a recall value of 0.830, an f-measure of 0.860 and an AUC value of 0.974 during the test. While using the Le-Net model, the results obtained are far lower than even the worst outcomes obtained using the hybrid model. The time windows used to execute training is another significant distinction between the two models. In particular, the Hybrid QCNN model executes the training phase takes much more time than the Le-Net despite the size of the image employed being almost the same. Of course, the training time has been used only to compare the models and is not to be considered to evaluate their performances. In addition, as reported at the beginning of this section, the experiments executed using the Hybrid QCNN were not carried out using a quantum computer. For this reason, the results obtained may have been degraded. Figure 2 shows the accuracy obtained during the training and validation, while in Figure 3 values of loss obtained in the same phases are reported. Performing other experiments, we can see that by raising the number of epochs or the batch size accuracy decrease. The latter phenomenon does not occur with the Le-Net, in that 30 epochs are required during the training test to achieve the best accuracy. Regarding the categorization obtained using the best settings of the Hybrid model, its confusion matrix in Figure 4 is reported. In addition to the trust samples, the model has a hard time recognizing elements belonging to the *Salinity* family. Specifically, in the latter family, the Hybrid QCNN network reached an accuracy value of 0.960, a precision of 0.428, a recall value of 0.235, an f-measure of 0.303, and an AUC value of 0.611 (True Positive: 12, True Negative: 1372, False Positive: 16, False Negative: 39). On the other hand, the model can distinguish perfectly samples belonging to the families named *Adposhel*, *Fasong*, *HackKMS*, *Regrun*, *Snarasite*, and *VBA*. In particular, on them, the model was able to reach values of accuracy, precision, recall, f-measure, and AUC of 1.0. The results obtained from the Hybrid QCNN model are better than the Le-Net model introduced in 1998. Although we used a smaller image size ($25 \times 25 \times 1$) to train the model than the convolutional model ($32 \times 32 \times 1$) due to the limitation of model complexity and computational resources required, we can more strongly state that quantum models are promising. Finally, in the future, the progress of computing will considerably enhance quantum networks.

B. GradCam++ and ScoreCam application

As the last step of the method we propose, shown in Figure 1, we employed two different CAM algorithms such as GradCam++ and ScoreCam. We applied them to the best

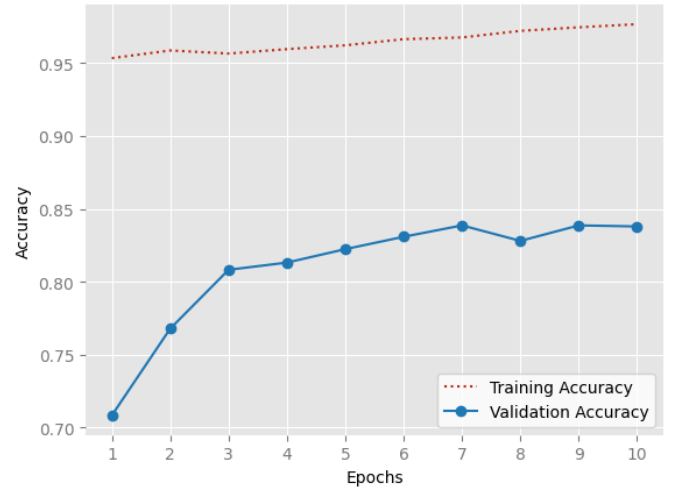


Fig. 2: Accuracy obtained in each epoch during the training and validation phases of the Hybrid QCNN model.

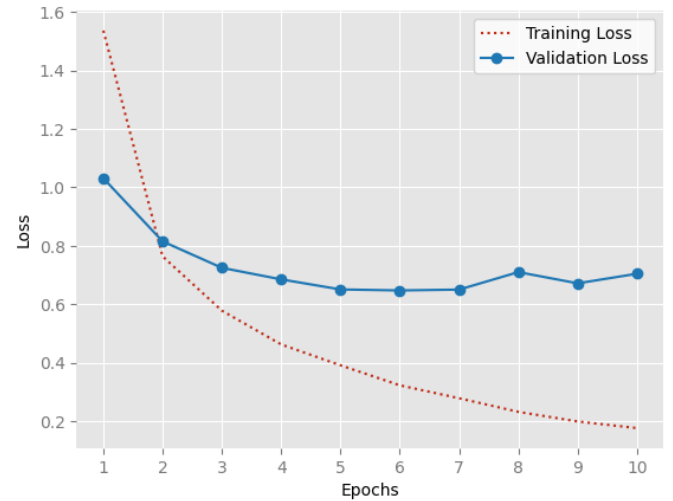


Fig. 3: Loss obtained in each epoch during the training and validation phases of the Hybrid QCNN model

settings obtained from the Hybrid QCNN model (according to Section IV-A). As a result of the application of CAMs, we retrieved several images in PNG format where the activation map is overlayed to the original image. The heatmaps are employed to take into account three distinct shadows: yellow, green, and blue. Precisely, areas over-shadowed with the color blue represent areas of disinterest to the model. Instead, the model is interested in the yellow-colored portions of the image. Green areas, on the other hand, are used to indicate central areas. Figures 5 and 6 are reported the GradCam++ application on the sample with the hash `0c8ab6ba97df07c366e9e9e26e27c477189caac7` belonging to the VBA family. As result, we can see that both algorithms achieved perfect classification, and the same areas were highlighted. While Figures 7 and 8 less precision of classification is reported.

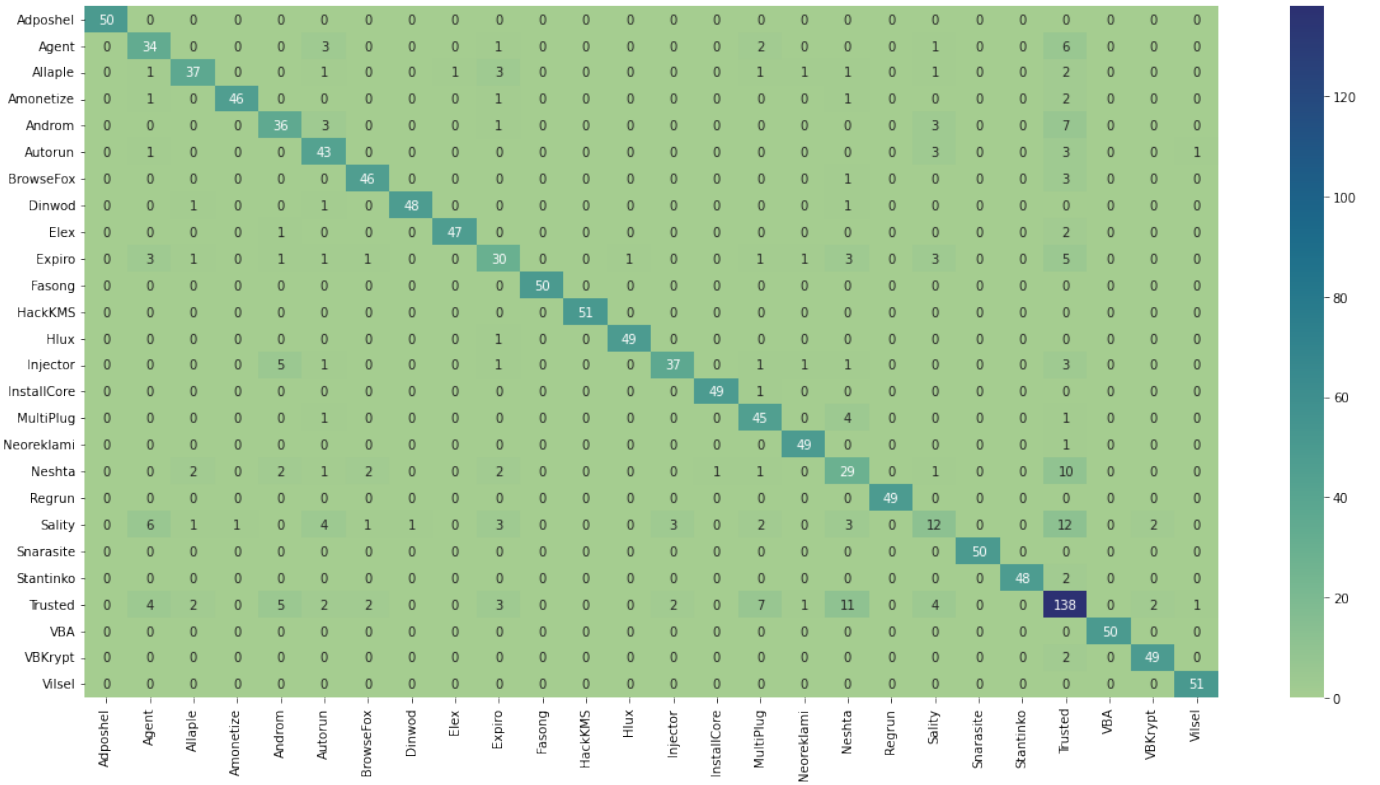


Fig. 4: Confusion Matrix

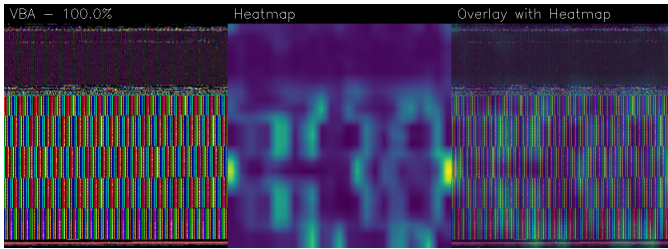


Fig. 5: Result of applying GradCam++ algorithm on sample 0c8ab6ba97df07c366e9e26e27c477189caac7

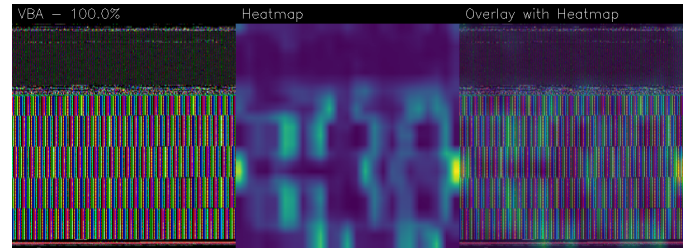


Fig. 6: Result of applying ScoreCam algorithm on sample 0c8ab6ba97df07c366e9e26e27c477189caac7

In particular, the algorithms applied on the sample 4bec58dbeabcf10db0a240c21a34a0c7f77ca8bc also in those images highlight the same areas and have the same result of the recognition of the sample. In conclusion, we can say the use of GradCam++ and ScoreCam algorithms is interesting for classification and distinguishing a group of images. In particular, thanks to the usage of explainability, users can determine which region of an image is most distinctive.

V. CONCLUSION AND FUTURE WORK

In this paper, we proposed a malware detector based on quantum computing by designing and evaluating the Hybrid-QCNN model. Furthermore, we analyze two different CAM algorithms to visually explain the model decision. We achieved interesting results obtaining an accuracy value of 0.850, a pre-

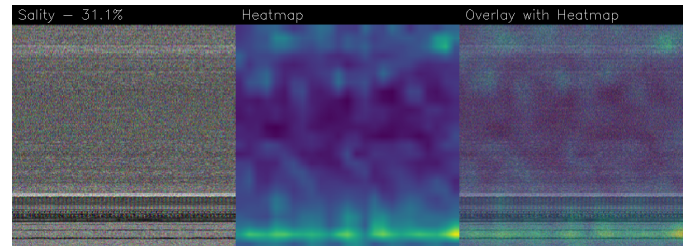


Fig. 7: Result of applying GradCam++ algorithm on sample 4bec58dbeabcf10db0a240c21a34a0c7f77ca8bc

cision of 0.892, a recall value of 0.830, an F-measure of 0.860, and an AUC value of 0.974. In addition, a comparison with one of the earliest CNN (i.e., Le-Net) is reported. We made this comparison to understand the evolution of Deep Learning

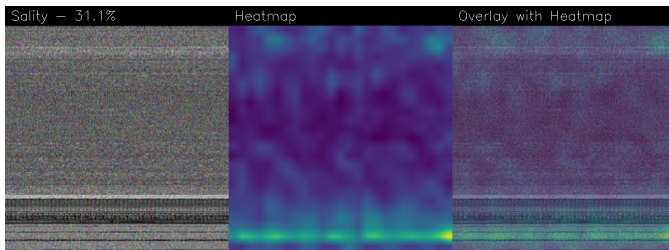


Fig. 8: Result of applying ScoreCam algorithm on sample 4bec58dbeabcf10db0a240c21a34a0c7f77ca8bc

models over the years and to demonstrate the (initials) quantum computing model can be really promising if compared to initial deep learning models. Finally, we demonstrated that despite being based on two distinct techniques, two different CAMs highlight the identical areas of an image obtained from malware under analysis. This allowed us to understand which portion of an image is most distinctive during the classification. In future studies, we intend to consider other models to perform a better comparison with other quantum models and more CAMs to perform a deeper analysis.

ACKNOWLEDGMENT

This work has been partially supported by EU DUCA, EU CyberSecPro, and EU E-CORRIDOR projects and PNRR SERICS_SPOKE1_DISE, RdS 2022-2024 cybersecurity.

REFERENCES

- [1] M. G. Cimino, N. De Francesco, F. Mercaldo, A. Santone, and G. Vaglini, "Model checking for malicious family detection and phylogenetic analysis in mobile environment," *Computers & Security*, vol. 90, p. 101691, 2020.
- [2] A. Cimitile, F. Martinelli, and F. Mercaldo, "Machine learning meets ios malware: Identifying malicious applications on apple environment," in *ICISSP*, 2017, pp. 487–492.
- [3] A. K. Sahoo, K. S. Sahoo, and M. Tiwary, "Signature based malware detection for unstructured data in hadoop," in *2014 International Conference on Advances in Electronics Computers and Communications*. IEEE, 2014, pp. 1–6.
- [4] M. Zheng, M. Sun, and J. C. Lui, "Droid analytics: a signature based analytic system to collect, extract, analyze and associate android malware," in *2013 12th IEEE International Conference on Trust, Security and Privacy in Computing and Communications*. IEEE, 2013, pp. 163–171.
- [5] G. Canfora, F. Mercaldo, C. A. Visaggio, and P. Di Notte, "Metamorphic malware detection using code metrics," *Information Security Journal: A Global Perspective*, vol. 23, no. 3, pp. 57–67, 2014.
- [6] M. Christodorescu and S. Jha, "Static analysis of executables to detect malicious patterns," Wisconsin Univ-Madison Dept of Computer Sciences, Tech. Rep., 2006.
- [7] F. Martinelli, F. Mercaldo, A. Saracino, and C. A. Visaggio, "I find your behavior disturbing: Static and dynamic app behavioral analysis for detection of android malware," in *2016 14th Annual conference on privacy, security and trust (PST)*. IEEE, 2016, pp. 129–136.
- [8] M. Ahmadi, D. Ulyanov, S. Semenov, M. Trofimov, and G. Giacinto, "Novel feature extraction, selection and fusion for effective malware family classification," in *Proceedings of the sixth ACM conference on data and application security and privacy*, 2016, pp. 183–194.
- [9] B. Sun, Q. Li, Y. Guo, Q. Wen, X. Lin, and W. Liu, "Malware family classification method based on static feature extraction," in *2017 3rd IEEE International Conference on Computer and Communications (ICCC)*. IEEE, 2017, pp. 507–513.
- [10] M. Schuld, I. Sinayskiy, and F. Petruccione, "An introduction to quantum machine learning," *Contemporary Physics*, vol. 56, no. 2, pp. 172–185, 2015.
- [11] G. Ciaramella, G. Iadarola, F. Mercaldo, M. Storto, A. Santone, and F. Martinelli, "Introducing quantum computing in mobile malware detection," in *Proceedings of the 17th International Conference on Availability, Reliability and Security*, 2022, pp. 1–8.
- [12] A. Chattopadhyay, A. Sarkar, P. Howlader, and V. N. Balasubramanian, "Grad-cam++: Generalized gradient-based visual explanations for convolutional networks," in *2018 IEEE winter conference on applications of computer vision (WACV)*. IEEE, 2018, pp. 839–847.
- [13] H. Wang, Z. Wang, M. Du, F. Yang, Z. Zhang, S. Ding, P. Mardziel, and X. Hu, "Score-cam: Score-weighted visual explanations for convolutional neural networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, 2020, pp. 24–25.
- [14] J. Amin, M. Sharif, N. Gul, S. Kadry, and C. Chakraborty, "Quantum machine learning architecture for covid-19 classification based on synthetic data generation using conditional adversarial neural network," *Cognitive Computation*, pp. 1–12, 2021.
- [15] F. Mercaldo, G. Ciaramella, G. Iadarola, M. Storto, F. Martinelli, and A. Santone, "Towards explainable quantum machine learning for mobile malware detection and classification," *Applied Sciences*, vol. 12, no. 23, p. 12025, 2022.
- [16] S. Ni, Q. Qian, and R. Zhang, "Malware identification using visualization images and deep learning," *Computers & Security*, vol. 77, pp. 871–885, 2018.
- [17] B. Cakir and E. Dogdu, "Malware classification using deep learning methods," in *Proceedings of the ACMSE 2018 Conference*, 2018, pp. 1–5.
- [18] J. Saxe and K. Berlin, "Deep neural network based malware detection using two dimensional binary program features," in *2015 10th international conference on malicious and unwanted software (MALWARE)*. IEEE, 2015, pp. 11–20.
- [19] K. He and D.-S. Kim, "Malware detection with malware images using deep learning techniques," in *2019 18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*. IEEE, 2019, pp. 95–102.
- [20] M. Broughton, G. Verdon, T. McCourt, A. J. Martinez, J. H. Yoo, S. V. Isakov, P. Massey, R. Halavati, M. Y. Niu, A. Zlokapa *et al.*, "Tensorflow quantum: A software framework for quantum machine learning," *arXiv preprint arXiv:2003.02989*, 2020.